

LOGIAGENT: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents

Ke Zhang*
Xidian University
China

Chenxi Zhang*
Xidian University
China

Chong Wang†
Nanyang Technological University
Singapore

Chi Zhang
Fudan University
China

YaChen Wu
Fudan University
China

Zhenchang Xing
CSIRO's Data61
Australia

Yang Liu
Nanyang Technological University
Singapore

Qingshan Li
Xidian University
China

Xin Peng
Fudan University
China

ABSTRACT

Automated testing for REST APIs has become essential for ensuring the correctness and reliability of modern web services. While existing approaches primarily focus on detecting **server crashes** and error codes, they often overlook logical issues that arise due to evolving business logic and domain-specific requirements. To address this limitation, we propose LOGIAGENT, a novel approach for **logical testing** of REST systems. Built upon a large language model (LLM)-driven multi-agent framework, LOGIAGENT integrates a Test Scenario Generator, API Request Executor, and API Response Validator to collaboratively generate, execute, and validate API test scenarios. Unlike traditional testing methods that focus on status codes like 5xx, LOGIAGENT incorporates **logical oracles** that assess responses based on business logic, ensuring more comprehensive testing. The system is further enhanced by an Execution Memory component that stores historical API execution data for contextual consistency. We conduct extensive experiments across **12 real-world REST systems**, demonstrating that LOGIAGENT effectively identifies **234 logical issues** with an accuracy of **66.19%**. Additionally, it basically excels in detecting **server crashes** and achieves superior test coverage compared to four state-of-the-art REST API testing tools. An ablation study confirms the significant contribution of LOGIAGENT's memory components to improving test coverage.

1 INTRODUCTION

REST (Representational State Transfer) has become the dominant architectural style for web services, providing a scalable and flexible approach to system integration through stateless interactions and standardized HTTP methods [12]. REST APIs enable interoperability across distributed systems, forming the backbone of modern cloud computing platforms, microservices architectures, and the Web of Things [4, 17, 32]. However, ensuring the correctness, robustness, and reliability of REST systems remains a significant challenge due to evolving API specifications, complex dependencies, and the potential for inconsistent behaviors across different execution environments [25, 26, 31, 42].

Automated REST API testing has become a vital solution to address these challenges, with recent research investigating methods that leverage large language models (LLMs) to improve test case generation [6, 22, 23]. RestGPT [22] and AutoRestTest [23] leverage LLMs to identify constraints and generate valid parameter values. RESTSpecIT [6] utilizes LLMs to analyze API responses to automatically infer API specification in addition to parameter generation. These works alleviate the problem of traditional methods that frequently generate invalid parameters. Despite the advances, they struggle to accurately capture real-world business logic, focusing on detecting 5xx errors while overlooking logical issues.

To this end, we propose conducting **logical testing** for REST systems to expand the scope of existing research on automated REST API testing. Unlike traditional testing objectives (e.g., *fuzz testing* for **server crashes**), logical testing seeks to address two key limitations. First, rather than directing test generation towards deeper exploration based on 404 or 5xx feedback, logical testing focuses the exploration on more complex, realistic scenarios guided by business logic feedback. Second, the validation of API request responses should not only consider status codes like 5xx, but also include **logical oracles** that reflect the business logic and domain knowledge. For example, some API requests may return responses with a 200 OK status, but the information in the response body may be logically incorrect given the specific scenario context.

In this paper, we introduce LOGIAGENT, a novel automated logical testing approach for REST systems based on an LLM-driven multi-agent system. The framework consists of three core agents: Test Scenario Generator, API Request Executor, and API Response Validator, which collaboratively generate, execute, and validate API test scenarios. A global Scenario Scheduler manages the execution status, while a long-term Execution Memory stores historical API execution data to maintain synchronization and contextual consistency. The process begins with the Test Scenario Generator, which creates a test scenario and adds it to the Scenario Scheduler. The workflow then cycles between the API Request Executor, which retrieves and executes test steps, and the API Response Validator, which validates the responses. Based on the validation results, the Scenario Scheduler updates the status and checks for scenario termination. Once terminated, the Test Scenario Generator creates a new scenario, and the process repeats.

*Both authors contributed equally to this work.

†Chong Wang is the corresponding author.

We conduct extensive experiments to evaluate the effectiveness of LOGIAGENT in detecting both logical errors and **server crashes** across **12 real-world REST systems**. The results show that LOGIAGENT effectively detects logical issues, identifying 234 issues (**139 bugs and 95 enhancements**) with a reasonable accuracy of **66.19%**. Additionally, LOGIAGENT demonstrates leading performance in detecting 500-code **server crashes** (**49 errors** identified in total) and achieves superior test coverage, compared to four state-of-the-art REST API testing tools. Finally, the results of an ablation study confirm the contribution of the memory components in LOGIAGENT to improved test coverage.

To summarize, this paper makes the following key contributions:

- LOGIAGENT, a novel automated logical testing approach for REST systems based on an LLM-driven multi-agent system, which extends the scope of existing research on automated REST API testing. The system includes three LLM-based agents, a global Scenario Scheduler, and long-term Execution Memory to collaboratively execute the testing workflow.
- Experimental results demonstrating the effectiveness of LOGIAGENT in detecting both logical issues and **server crashes** in REST systems. It identifies **234 logical issues** and **49 server crashes** across **12 real-world REST systems**.

2 PRELIMINARIES AND MOTIVATION

In this section, we first introduce the fundamentals of logical testing for REST systems and contrast it with traditional REST API fuzz testing. We then present motivating examples to highlight the challenges involved in logical testing for REST systems.

2.1 Logical Testing for REST Systems

Business Scenario Orchestration. In logical testing, the goal is to model real-world workflows and use cases that the APIs are designed to support, simulating practical business scenarios to ensure that APIs are called as intended and validated under real-world conditions. While traditional fuzz testing also generates API call chains, its primary focus is on creating diverse API request parameters and triggering various and deep execution paths, without guaranteeing that these paths align with the way real users interact with the system. Moreover, fuzz testing typically sends random or edge-case data to identify problems like **server crashes** or failures (e.g., 5xx error codes), whereas logical testing aims to uncover logical issues that extend beyond **server crashes**. For example, in an e-commerce platform, a business scenario might involve steps such as creating a user profile, adding products to a shopping cart, placing an order, and processing payment. Logical testing ensures that these steps are executed as orchestrated, verifying that the API functions consistently and as intended across multiple interconnected endpoints, while adhering to real-world business logic. This requires a higher level of orchestration of REST APIs to uncover more realistic issues that would be triggered by general users.

Oracle and Response Validation. In logical testing, an oracle defines the expected behavior or output of an API in response to a given input. Unlike traditional fuzz testing, which typically focuses on basic checks (e.g., validating if a response code falls within a specific range like 5xx), logical testing requires more advanced oracles. These oracles go beyond checking the response code; they

assess the content, structure, and relationships within the response to ensure they align with the intended business logic. The oracle is derived not only from the API's technical specifications but also from a deep understanding of the semantic and logical relationships embedded in business rules, domain knowledge, and use-case scenarios. Effective response validation, based on these oracles, ensures that the API functions as intended under various test conditions, identifying logical discrepancies that might otherwise be overlooked in traditional crash-focused fuzz testing. For example, when creating a user profile through an API, the oracle defines the expected data format (e.g., name, age, address) and ensures that the response aligns with the semantic meaning of the data. This involves checking that the returned data accurately reflects the input and adheres to business rules—such as ensuring the user's age is logically consistent with other fields (e.g., age should not be negative or unrealistic) and that the user's address conforms to valid real-world standards (e.g., proper address format).

2.2 A Motivating Example

We use the PetStore system [35] as an example to illustrate the motivation behind logical testing and highlight key challenges. Consider a multi-step business scenario: a user registers, logs in, creates a pet profile, deletes the pet, and then attempts to place an order for the deleted pet. Executing these API calls in PetStore reveals three logical issues.

First, when creating a pet profile, the "POST /pet" API (Figure 1a) is used to add a new pet with details such as category, name, photo URLs, and tags. However, sending a request with an invalid photo URL (e.g., "example") results in a 200 OK response (Figure 1b), indicating successful execution. This exposes two issues: (1) By REST design conventions, a successful creation via POST should return 201 Created rather than 200 OK [11]; (2) The system incorrectly accepts invalid photo URL, leading to potential data integrity and accessibility problems. A more severe logical issue arises in the final step when creating an order for a deleted pet. Despite the pet no longer existing, the system incorrectly returns a 200 OK response, causing a logical inconsistency that may lead to critical vulnerabilities.

This example highlights three key insights. First, logical testing is essential for identifying both minor inconsistencies and severe business logic flaws in REST systems. Second, certain logical flaws only emerge through carefully constructed sequences of operations, underscoring the need for complex and coherent test scenarios. Third, validating API responses requires robust oracles derived from multiple sources beyond API documentation, including business logic, domain knowledge, and best practices.

3 APPROACH

In this paper, we propose LOGIAGENT, an approach to automated logical testing for REST systems using a multi-agent system powered by large language models.

3.1 Overview

As shown in Figure 2, the system comprises three primary LLM-based agents: **Test Scenario Generator**, **API Request Executor**,

POST /pet Add a new pet to the store

Parameters Try it out

Name	Description
body ★ required	Pet object that needs to be added to the store

Example Value | Model

```
{
  "id": 0,
  "category": {
    "id": 0,
    "name": "string"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Parameter content type: application/json

Responses Response content type: application/json

Code	Description
405	Invalid input

(a) The OpenAPI Specification

Server response

Code	Details
200	Response body <pre>{ "id": 1, "category": { "id": 1, "name": "husky" }, "name": "doggie", "photoUrls": ["example"], "tags": [{ "id": 1, "name": "boydog" }], "status": "available" }</pre>

(b) An Example Response

Figure 1: The OpenAPI Specification and Example Response of /pet API

and **API Response Validator**. These agents collaboratively generate, execute, and validate API test scenarios. Additionally, a global **Scenario Scheduler** maintains the execution status of the current test scenario, while a long-term **Execution Memory** stores historical API request executions, ensuring synchronized scheduling and contextual consistency across agents. In this section, we first introduce the key components of the multi-agent system before detailing the overall workflow. The details of the three agents are introduced in the following sections.

3.1.1 LLM-based Agents. The following describes the roles and functionalities of the three LLM-based agents:

- **Test Scenario Generator** leverages an LLM to create comprehensive and diverse test scenarios that accurately reflect the business logic of the target REST system. Each time it runs, the generator prompts an LLM to produce a new test scenario consisting of a sequence of **Test Steps**, each paired with its expected behavior as defined by **Test Oracles**. A test step includes an API to execute and a brief description, while an oracle is derived from multiple sources, including API documentation (e.g., expected response descriptions), the scenario context (e.g., business logic), and general LLM knowledge (e.g., best practices beyond the API documentation). To ensure that generated scenarios align with real-world business logic, the generator utilizes the **API Relationship Graph (ARG)**, an undirected graph constructed from API documentation that encodes semantic relationships between APIs. Additionally, to promote diversity and minimize redundancy, it considers previously generated test scenarios and their execution results stored in the Execution Memory.
- **API Request Executor** is responsible for constructing request payloads (API parameters) for a test step using an LLM and executing the API request. To maintain contextual consistency (e.g., ensuring the same user ID across API requests), the executor considers previous API requests and responses within the same scenario (i.e., a short memory specific to the current scenario) before prompting the LLM. Additionally, it retrieves relevant historical execution information from the long-term Execution Memory, including reference parameter values and prior failure reflections. This helps the agent avoid known valid or invalid parameter values when constructing requests. Once the API request is prepared, the executor invokes the `do_request` function to execute it and collects the response.
- **API Response Validator** employs an LLM to assess whether the collected API response aligns with the expected behavior (i.e., oracle) based on the current scenario context (e.g., business logic and execution state) and the LLM’s text understanding capabilities. If the response violates the oracle, an issue—ranging from **server crashes** to logical inconsistencies—is reported. The API request details and failure reflections (i.e., explanations generated by the LLM) are also recorded in the Execution Memory to guide future request optimizations.

3.1.2 Scenario Scheduler. When testing a target REST system, a key challenge is managing the execution status of scenarios and steps while coordinating the execution flow among agents. Specifically, the multi-agent system must determine when a scenario has been fully executed or has failed, requiring the generation of a new scenario. It also needs to assess whether a specific step has passed or requires re-execution. To address this, we design a global Scenario Scheduler to enhance status awareness within the multi-agent system. The scheduler maintains a list of test steps and a pointer to the current step being executed. Each step has a `retry` property indicating the number of re-execution attempts.

- **Scenario Addition (`add_scenario`):** Each time the Test Scenario Generator creates a new test scenario, its steps are sequentially added to the scheduler. The pointer is set to the first step, and each step’s `retry` count is initialized to zero.

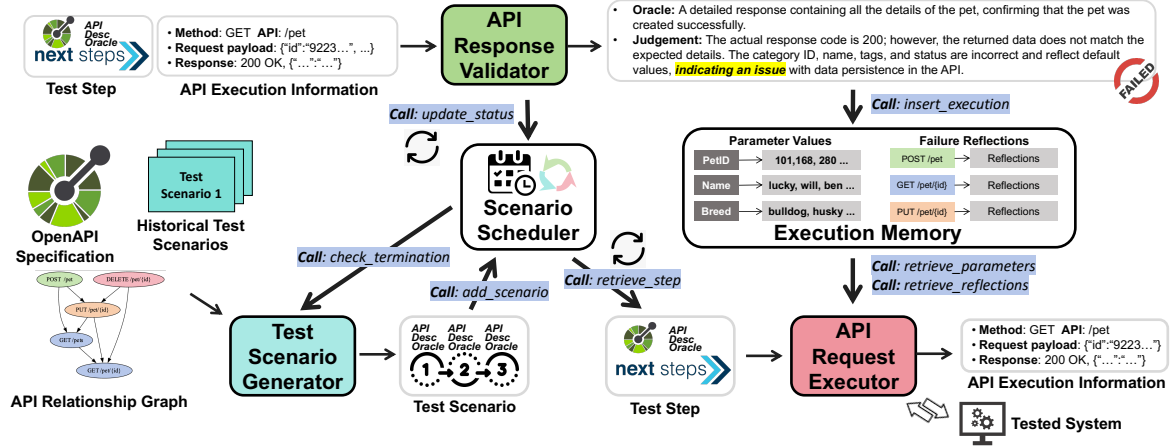


Figure 2: Approach Overview of LOGIAGENT

- **Step Retrieval (*retrieve_step*)**: The scheduler returns the step currently pointed to. Notably, the retrieved step may be the same as in the previous retrieval if a failure occurred, necessitating a step re-execution.
- **Status Update (*update_status*)**: After validating the current API request, the scheduler updates the execution status. If the validation confirms that the step has passed, it is removed from the step list, and the pointer advances to the next step. Otherwise, the *retry* count is incremented. If *retry* exceeds a predefined limit (e.g., 3), the step list is cleared, and the scenario is marked as a failed execution.
- **Termination Check (*check_termination*)**: Determines whether the current scenario should be terminated and a new one generated. Termination is triggered when the step list becomes empty.

3.1.3 Execution Memory. To ensure sufficient exploration depth and test coverage, it is crucial for the API Request Executor to generate valid and diverse parameter values. However, LLMs struggle to produce such inputs effectively without execution feedback. To address this, we introduce an Execution Memory to facilitate information sharing between the API Request Executor and API Response Validator. Specifically, this memory maintains records of: (i) parameter names and values (e.g., *product_status*=*'sold'*) for requests whose responses successfully pass validation, and (ii) failure reflections for requests whose responses fail validation, where the reflections are generated by LLMs in the API Response Validator.

- **Feedback Insertion (*insert_execution*)**: Each time the API Response Validator completes validation for an API response, execution feedback is inserted into memory. If the validation is passed, the parameter names and values from the API request are stored as key-value pairs. Otherwise, a failure reflection record is inserted, where the API serves as the key, and the corresponding API request details and failure explanation are stored as the value.
- **Parameters Retrieval (*retrieve_parameters*)**: Given an API, the BM25 (Best Matching 25) retrieval algorithm [34] filters relevant records from memory to provide reference values for its parameters. The retrieval query combines the API specification's description text with the current step description, retrieving the

top 10 most relevant previously stored parameter names and values. Notably, the retrieved parameters are not necessarily from the current API being executed, as semantically similar parameter names across different APIs enable a degree of parameter value generalization.

- **Reflections Retrieval (*retrieve_reflections*)**: Given an API, this function returns stored failure reflection records of this API. These reflections help accumulate system knowledge over time, reducing the likelihood of repeating the same errors and improving the effectiveness of future executions.

3.1.4 Workflow. As illustrated in Figure 2, given a target REST system, the process begins with the Test Scenario Generator, which generates a test scenario and adds it to the Scenario Scheduler by calling the *add_scenario* function. Next, the workflow enters a cycle between the API Request Executor and the API Response Validator to execute and validate the scenario. Specifically, the API Request Executor continuously retrieves the current step to execute from the Scenario Scheduler by calling the *retrieve_step* function, executes the step, and collects the response. The collected response is then passed to the API Response Validator for validation. Based on the validation results, the Scenario Scheduler updates the execution status by calling the *update_status* function and checks whether the current scenario is terminated using the *check_termination* function. This cycle repeats until the *check_termination* function returns true, at which point the Test Scenario Generator is reactivated to generate a new test scenario, and the Scenario Scheduler begins managing the newly generated scenario.

3.2 Test Scenario Generator

The Test Scenario Generator creates test scenarios by leveraging an LLM to analyze API documentation and apply its acquired domain knowledge, allowing for a deeper understanding of the business logic behind the target REST system.

3.2.1 API Relationship Graph. While the API documentation (OpenAPI Specification Format ¹) provides detailed descriptions of individual APIs, its raw organizational structure makes it challenging

¹<https://swagger.io/specification/>

for LLMs to capture the semantic relationships between them. The documentation typically presents APIs as isolated entities, each with a brief functionality description, parameter details (with examples), and response information. However, real-world business scenarios often involve sequences of API calls that encode underlying semantic logic. To bridge this gap, we introduce the **API Relationship Graph (ARG)**, an undirected graph that models API interactions and dependencies, enhancing the global understanding of APIs within the target REST system.

To construct the ARG, we employ a two-phase process to identify semantically related API pairs. In the first phase, we iterate through all possible API pairs and compute their semantic similarity. Each API is encoded into a vector representation using the OpenAI embedding model [33], which processes the API's description text. The semantic similarity between two APIs is then measured using cosine similarity, and only pairs that exceed a predefined threshold (e.g., 0.5) are retained. In the second phase, for each retained API pair, an LLM (e.g., **GPT-4o-mini**) analyzes their API documentation to determine whether they exhibit logical interactions or dependencies. Through this process, the ARG is constructed by representing APIs as nodes and inserting edges between related API pairs. Note that if isolated APIs exist, we connect them to all other APIs by adding edges. Each API node in the ARG is further enriched with attributes such as request method (e.g., GET and POST), path, and a structured Markdown-formatted description. The first phase serves as a necessary filtering step, as when the number of APIs grows, making direct pairwise evaluation with LLMs impractical due to computational constraints.

Based on the constructed ARG, we apply **random walk** (maximum length is 10) to dynamically select a subset of interconnected APIs that capture semantic aspects of the target REST system. Compared to directly feeding all APIs into LLMs for test scenario generation, this approach exposes LLMs to diverse API combinations while ensuring focus on relevant APIs and reducing context overload.

3.2.2 Agent Initialization. The prompt template shown in Figure 3 is designed to initialize the LLMs as high-quality test scenario generators. Note that the prompt has been simplified for clarity, while the complete version is available in our replication package [1].

Specifically, the prompt template comprises four key components: **Role Assignment**, which defines the LLM's persona as a specialized agent for generating test scenarios; **Detailed Requirements**, which outlines specific criteria to ensure the generation of high-quality test scenarios. For a REST system, an effective test scenario should not only simulate real-world business logic but also introduce new situations that have not yet been tested. To achieve this, we encode these requirements into the instructions and specify a structured output format where each step includes a Title, API, Description, and Expected Response (Oracle); **Few-shot Examples**, which provide carefully crafted example test scenarios inspired by in-context learning [7, 30], helping the LLM understand the task requirements and expected format; and **Input Placeholders**, which supplies a set of APIs selected through random walk on the ARG, along with previously generated test scenarios, to prevent redundancy and encourage diversity in scenario generation.

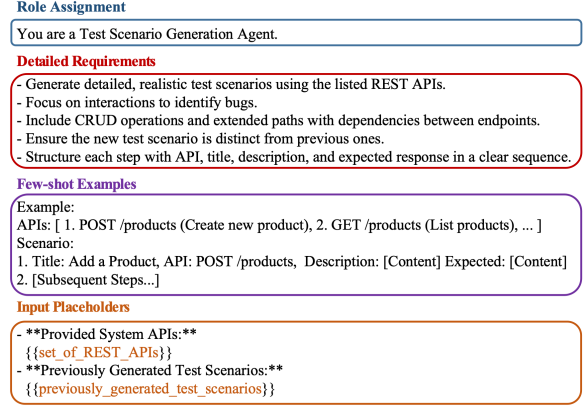


Figure 3: Prompt Template Used in Test Scenario Generator

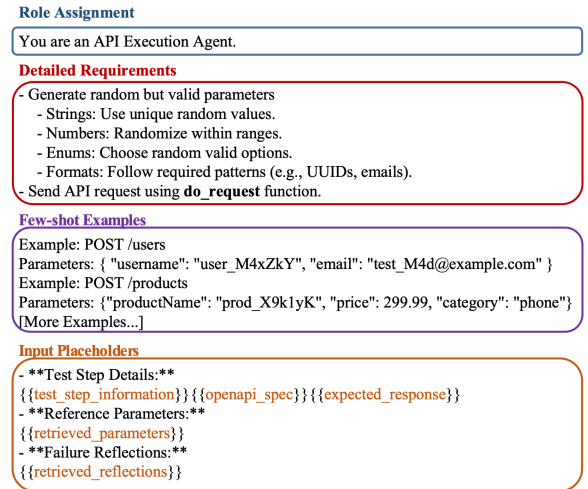


Figure 4: Prompt Template Used in API Request Executor

3.2.3 Test Scenario Generation. Each time the agent executes, the input placeholders in the prompt template are filled with the required inputs. First, a set of relevant APIs is collected by applying random walk on the ARG and inserted into the corresponding placeholder. Then, historical test scenarios are fed into the corresponding placeholder. To ensure the context window size is within limits, no more than ten historical scenarios are provided, selected randomly. Once the inputs are filled into the prompt template, the agent requests the LLMs to generate a structured test scenario, typically consisting of 8 to 12 test steps. Each step includes the relevant API information, a brief description of the step, and the expected response of the request (i.e., oracle). The generated scenario is subsequently sent to the Scenario Scheduler.

3.3 API Request Executor

The API Execution Agent is the agent responsible for generating API request payload and executing actual API request.

3.3.1 Agent Initialization. This executor agent is initialized using a four-component prompt template, as shown in Figure 4. **Role Assignment** defines the agent's primary function as an API execution

handler; **Detailed Requirements** specify the API request execution using *do_request* function and parameter generation rules for various data types with their validation patterns; **Few-shot Examples** demonstrate well-designed API request parameters examples, helping the LLM understand how to generate random and valid parameters for APIs; and **Input Placeholders** supply the necessary details for executing each step.

3.3.2 Test Step Retrieval. The agent first utilizes the *retrieve_step* function tool provided by the Scenario Scheduler to retrieve the next step for execution, which may be the same as in the previous iteration if a verification failure occurred.

The retrieved test step consists of three primary components, with historical information gathered from Execution Memory:

- **Test Step Details:** This section includes the test step name, API, test description, API specifications in Markdown format, and the expected response. It serves as the fundamental context for executing the current test step. Notably, the expected response (i.e., the oracle) is also provided as auxiliary information to help the LLM understand the step's intent.
- **Reference Parameters:** To generate parameter values efficiently and effectively, relevant historical parameter values are retrieved from the Execution Memory by calling its *retrieve_parameters* function as references.
- **Failure Reflections:** Similarly, relevant failure reflection information is retrieved from the Execution Memory by calling the *retrieve_reflections* function to guide the LLM in generating valid parameters, improving the likelihood of a successful API request and enhancing exploration depth.

The retrieved information is then filled into Input Placeholders to construct a concrete prompt.

3.3.3 Request Execution. After generating the parameters, the agent constructs the API payload and sends the request to the target REST system using the *do_request* function call, then collects the response. The *do_request* function is implemented as a Python tool that provides a clean interface for the agent.

3.4 API Response Validator

After each request execution, the API Response Validator evaluates whether the response aligns with expectations and provides an explanation for its judgment.

3.4.1 Agent Initialization. As shown in the figure 5, the agent's prompt template consists of four components:

Role Assignment defines the agent's primary function as an API Response Validator; **Detailed Requirements** specify the validation process through response analysis from multiple perspectives and define the validation criteria for response assessment; **Few-shot Examples** demonstrate typical validation scenarios with clear alignment indicators, helping the LLM understand how to assess different types of response mismatches; and **Input Placeholders** supply the necessary context including test step details and actual request-response data for validation.

3.4.2 Validation Process. In the validation process, the agent first fills required information into **Input Placeholders** in the prompt template. To ensure clarity, this not only provides the API response

Role Assignment

You are an API Validation Agent.

Detailed Requirements

- Validate results based on the response code and key elements in response body.
- Analyze responses through multiple perspectives before making final decisions, ensuring comprehensive validation coverage.
- Responses with major mismatches are marked as unaligned with detailed explanations, while minor mismatches are still marked as aligned but highlighted in comments.

Few-shot Examples

Example Validations, (✓ indicates aligned, ✗ indicates not aligned):

- ✗ "Missing required 'id' field for resource identification"
- ✗ "Deleted resource still accessible, violating expected state"
- ✓ "Additional field 'timestamp' found but core data intact"

[More Examples...]

Input Placeholders

```

- **Test Step Details:**
{{test_step_information}} {{openapi_spec}} {{expected_response}}
- **Actual Request & Response:**
{{request_data}} {{response_data}}

```

Figure 5: Prompt Template Used in API Response Validator

code and response body, but also provides the current API information in Markdown format, along with the request payload and parameters during request execution. This guarantees that the evaluation is based on complete and structured context, facilitating accurate validation.

After gathering all necessary context, the API Validation Agent employs a multi-perspective analysis approach when validating responses. By evaluating multiple aspects of the response simultaneously, the agent ensures comprehensive validation coverage. This approach leads to more thorough validation results, where responses with major mismatches are marked as unaligned with detailed explanations, while minor mismatches are documented in the reasoning process.

If the agent detects any issues, such as **server crashes** or logical inconsistencies, it reports them. The *Execution Memory* is then updated based on the validation results by invoking the *insert_execution* function.

4 EVALUATION

We conduct extensive experiments to evaluate the effectiveness of the proposed LOGIAGENT in testing real-world REST systems.

First, we run LOGIAGENT and 4 state-of-the-art baselines on **12 real-world REST systems** and record the reported problems, including both *logical issues* and **server crashes**. We assess LOGIAGENT's precision in detecting logical issues through manual annotation (RQ1) and evaluate its effectiveness in detecting **server crashes** by comparing it against the baselines (RQ2). Second, we analyze the test coverage achieved by LOGIAGENT on the tested REST systems and compare it with the baselines (RQ3). Third, we perform an ablation study to investigate the contribution of the memory components in LOGIAGENT (RQ4).

All research questions are listed as follows:

- **RQ1 (Effectiveness in Detecting Logical Issues):** How effectively does LOGIAGENT identify logical issues?
- **RQ2 (Comparison in Detecting Server Crashes):** How does LOGIAGENT perform in detecting **server crashes** (500 Internal Server Error) compared to the baselines?

Table 1: REST Systems Used in the Evaluation.

REST System	Lines of Code	#Operations
Features Service	1,646	18
Rest Countries	32,494	22
News Service	590	7
SCS	634	11
NCS	569	6
Language Tool	677,521	2
Person Controller	1,386	12
Project Track	88,634	67
User Management	5,108	22
Genome Nexus	22143	23
Person Controller	1,386	12
PetStore	Online	20
Bill Service	Online	21
Genome Nexus	Online	23

- **RQ3 (Test Coverage):** How does LOGIAGENT’s test coverage, including operation coverage and code coverage, compare to the baselines?
- **RQ4 (Ablation Study):** What is the impact of LOGIAGENT’s memory components on its performance?

4.1 Experimental Setup

4.1.1 Tested Systems. For evaluating LOGIAGENT, we select a diverse set of 12 REST systems, including both online and local deployments, to ensure a comprehensive assessment of our approach. As shown in Table 1, these systems vary significantly in size, ranging from 556 to 677,521 lines of code, and complexity, containing between 2 and 67 API operations.

Three widely recognized online systems are included to benchmark performance against established systems: Petstore [35], the official Bills API of the UK Parliament [37], and the publicly accessible Genome Nexus [14].

The remaining nine systems—Features-Service, REST Countries, News Service, SCS, NCS, Language Tool, Person-Controller, Project-Tracking-System, and User Management—are selected from a recent study [24] and deployed in our local environment. Note that we exclude ten REST systems tested in [24] from our evaluation due to various technical constraints. Four systems (Erc20 Rest Service, Spring Boot Actuator, Proxyprint, and Scout API) fail to provide reliable responses—two due to outdated dependencies incompatible with current external systems, and the other two because of authentication challenges that prevent meaningful testing. CatWatch and CWA Verification are omitted due to stringent rate-limiting mechanisms that significantly hinder the testing process and prevent timely result collection. Problem Controller and Spring Batch REST are excluded as their effective testing requires specialized domain expertise, which would introduce variables unrelated to our tool’s performance. Finally, OCVN and Market Service present substantial deployment difficulties that prevent their successful implementation in our testing environment.

This selection enables us to compare our results with existing research and evaluate LOGIAGENT’s effectiveness across a range of application domains.

4.1.2 Baselines. We select four state-of-the-art REST API testing tools as baselines to study the effectiveness of LOGIAGENT in detecting **server crashes** (i.e., 500 errors) and test coverage:

- RESTler [3]: A stateful REST API testing tool open sourced by Microsoft.
- EvoMaster [9]: A search-based Web API testing tool based on evolutionary algorithms. We use its black-box mode in our experiments.
- Morest [27]: A model-based testing tool that utilizes a dynamic property graph to guide API call sequence generation.
- ARAT-RL [21]: A reinforcement learning-based approach that adaptively prioritizes operations and parameters to optimize the exploration process in REST API testing.

Since these baselines tools were originally designed for long-running testing, we conduct two experimental settings for each: one with a fixed number of API requests (1,000 calls) and another with a fixed time budget (1 hour), ensuring a fair and comprehensive performance comparison.

4.1.3 Implementation. For our experiments, we use **GPT-4o-mini** with default hyperparameters as the foundational LLM for all agents in LOGIAGENT, striking an optimal balance between model capabilities and computational cost. Before starting the workflow for each tested system, we restart the agents, clear the Scheduler and Memory, and reset the databases to ensure consistent initial states and eliminate cross-contamination between runs. Each system is tested by LOGIAGENT with a standardized budget of 1,000 API requests to ensure consistent evaluation across all test subjects. The experiments are conducted on cloud servers equipped with 8-core, 16-thread Intel Xeon Platinum processors, 32GB of RAM, and running Ubuntu 22.04.5 LTS.

4.2 RQ1: Detection Effectiveness for Logical Issues

4.2.1 Method. For the potential logical issues reported by LOGIAGENT, manual verification is necessary, as they often rely on diverse oracles (e.g., format requirements) and cannot be validated using status codes alone, unlike **server crashes**.

For each logical issue report, we first collect all relevant information, including the request data, response data, the generated oracle, and LOGIAGENT’s reasoning for why the response fails to meet expectations. The manual verification process begins with an analysis of the complete test scenario context—specifically, the sequence of all requests and responses within the given test case. This contextual examination is crucial for distinguishing true issues from false positives, particularly when incorrect oracles result from LLM hallucinations. Human annotators then conduct an in-depth review of LOGIAGENT’s rationale to determine whether the reported issue is valid or a *false positive*. Note that during manual verification, we also eliminate duplicate reports that share the same API and issue type.

If the logical issue is confirmed as valid, we classify its severity into two categories: **Enhancements** and **Bugs**. Enhancements refer to minor deviations that do not critically affect functionality, such as non-standard HTTP response codes or response structures differing from OpenAPI specifications, which may stem from inconsistencies

Table 2: Effectiveness of LOGIAGENT in Detecting Logical Issues

Services	Total Reports	Bugs		Enhancements		All Issues		False-Positives	
	Count	Count	Rate	Count	Rate	Count	Rate	Count	Rate
Petstore	41	20	48.78%	11	26.83%	31	75.61%	10	24.39%
Bill-Service	23	7	30.43%	4	17.39%	11	47.83%	12	52.17%
Genome-Nexus	49	27	55.10%	3	6.12%	30	61.22%	19	38.78%
Features-Service	21	7	33.33%	8	38.10%	15	71.43%	6	28.57%
RESTCountries	43	24	55.81%	16	37.21%	40	93.02%	3	6.98%
News-Service	12	3	25.00%	5	41.67%	8	66.67%	4	33.33%
SCS	22	15	68.18%	5	22.73%	20	90.91%	2	9.09%
NCS	20	7	35.00%	7	35.00%	14	70.00%	6	30.00%
LanguageTool	23	12	52.17%	3	13.04%	15	65.22%	8	34.78%
Person-Controller	9	2	22.22%	2	22.22%	4	44.44%	5	55.56%
Project-Track	48	7	14.58%	17	35.42%	24	50.00%	24	50.00%
User-Management	38	8	21.05%	14	36.84%	22	57.89%	16	42.11%
Total Count / Average%	349	139	38.47%	95	27.71%	234	66.19%	115	33.81%

in the specifications themselves. In contrast, bugs indicate severe logical inconsistencies, such as stored data becoming inaccessible or system behavior significantly deviating from expected functionality.

4.2.2 Overall Results. Table 2 provides detailed results of LOGIAGENT’s effectiveness in detecting logical issues. In total, LOGIAGENT reports 349 potential logical issues, of which **234 (66.19%)** are confirmed as true system problems through rigorous manual verification. Specifically, **139 (38.47%)** are classified as logical **bugs**, while **95 (27.71%)** are identified as **enhancements** requiring system improvements. The remaining **115 (33.81%)** are determined to be false positives. These results demonstrate LOGIAGENT’s effectiveness, achieving a substantial detection accuracy of **66.19%** across diverse REST systems while maintaining a reasonable false positive rate.

4.2.3 Breakdown Analysis. Beyond the overall results, a more granular analysis uncovers key patterns based on different characteristics of the tested systems.

LOGIAGENT exhibits higher false-positive rates in complex, domain-specific systems with numerous APIs and intricate business logic. Notably, BillService (47.83% accuracy, 12 false positives) and Project-Track (50.00% accuracy, 24 false positives) illustrate this trend. Project-Track, with its 67 APIs, and BillService, which provides APIs for accessing and searching UK parliamentary bills and their legislative processes, highlight two key limitations of LLMs in handling domain-specific systems. First, maintaining a comprehensive understanding is challenging when dealing with multiple interconnected APIs and complex specifications. Second, limited domain-specific knowledge leads to misinterpretations of intricate business rules, contributing to the higher false-positive rates in these complex systems.

LOGIAGENT can achieve high detection accuracy in certain poorly documented systems. For example, RESTCountries (93.02% accuracy, 40 true issues), SCS (90.91% accuracy, 20 true issues), and NCS (70% accuracy, 14 true issues) exhibit high accuracy rates despite lacking details in their OpenAPI documentation, such as parameter specifications and response formats. This is due to the fact that

LLMs understand a large amount of generalized knowledge, which allows it to reason through parts of the documentation to obtain the functionality that the system should have. At the same time, combined with the feedback from the testing process, it can reason about the undocumented information. Therefore, LOGIAGENT can achieve a high accuracy even in the absence of documentation.

A particularly interesting case: LOGIAGENT identifies 15 true issues in LanguageTool, which has only 2 APIs. With only two APIs, LanguageTool focuses on language detection and grammar checking without complex business scenarios. However, its internal processing logic is highly intricate (677K LOC) due to the complexities of natural language analysis, particularly its multi-linguistic support. The relatively high number of detected issues likely stems from the LLM’s strong linguistic knowledge and language processing capabilities, enabling LOGIAGENT to generate diverse test inputs (e.g., multi-linguistic texts) and reliable oracles to assess LanguageTool’s correctness and consistency. This is further supported by the significantly higher code coverage LOGIAGENT achieves for LanguageTool compared to other baselines (see Figure 7).

4.3 RQ2: Detection Effectiveness for Server Crashes

4.3.1 Method. Server crashes are a critical type of unhandled failure in REST systems and serve as the primary detection objective of baseline methods. To assess LOGIAGENT’s effectiveness in identifying such failures, we systematically collect and analyze API responses that result in 500 status codes across LOGIAGENT and baseline approaches. Specifically, for each API, we extract all responses that trigger a 500 error and then deduplicate them based on response content.

4.3.2 Results. Table 3 compares server crash detection results of different approaches. LOGIAGENT achieves the highest effectiveness within the same request budget (1,000 requests). When baselines are allowed to send significantly more requests within a fixed time duration, most—except Morest—achieve comparable results to LOGIAGENT by generating a large volume of requests. For instance,

Table 3: Distinct Server Crashes Detected by Different Tools

Services	LOGIAGENT	ARAT-RL		EvoMaster		Morest*		RESTler	
	1000	1000	1h	1000	1h	1000	1h	1000	1h
Petstore	3	0	0	2	2	1	1	1	7
Bill-Service	1	0	0	2	2	1	1	0	0
Genome-Nexus	0	0	0	2	3	0	0	0	0
Features-Service	22	14	17	13	13	14	11	5	16
RESTCountries	0	1	1	1	1	1	1	0	1
News-Service	2	0	0	0	0	1	1	0	0
SCS	0	0	0	0	0	0	0	0	0
NCS	0	0	0	0	0	0	0	0	0
LanguageTool	1	0	0	0	0	0	0	0	0
Person-Controller	9	8	7	8	8	2	2	3	18
Project-Track	3	4	15	5	7	2	3	3	5
User-Management	8	5	5	6	7	6	6	6	7
Total	49	32	45	39	43	28	26	18	54

RESTler typically sends over 100K requests per hour for locally deployed systems, resulting in high system resource overhead and low exploration efficiency.

Notably, for Morest [27], 500 responses are deduplicated only based on APIs rather than response content, as its output does not include detailed response data.

4.4 RQ3: Test Coverage

4.4.1 Method. In this section, we compare the test coverage achieved by LOGIAGENT and the baselines, following prior studies [24, 28, 36, 41]. Test coverage is categorized into operation coverage and code coverage.

Operation coverage applies to all tested systems. By analyzing the requests and responses generated during execution, we count successfully executed API calls (i.e., those returning 2xx status codes) to assess operation coverage. This metric reflects how well a tool explores different APIs, providing a measure of its effectiveness in interacting with the tested system.

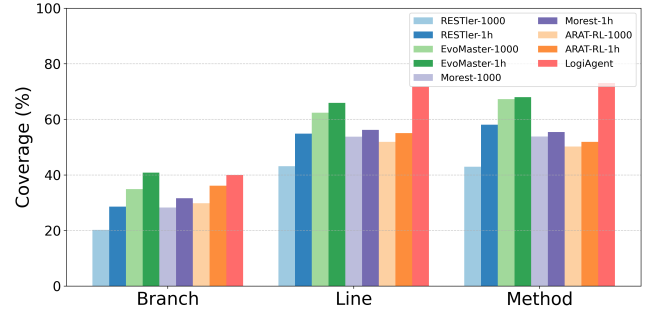
Code coverage applies only to REST systems that can be executed locally, as it requires inspecting their runtime behavior. We use JaCoCo [19] to collect coverage data during service execution, measuring branch, line, and method-level coverage for both LOGIAGENT and the baselines. Code coverage quantifies how thoroughly a tool exercises the service’s internal logic, offering insights into its ability to explore different execution paths.

4.4.2 Results. The overall operation coverage and code coverage are presented in Table 4 and Figure 6, demonstrating LOGIAGENT’s superior performance in achieving test coverage.

Figure 7 presents system-specific code coverage. Compared to baseline tools limited to 1,000 requests (ARAT-RL-1000, Morest-1000, EvoMaster-1000, and RESTler-1000), LOGIAGENT consistently achieves higher coverage across all metrics. On average, LOGIAGENT attains **39.98% branch coverage**, **71.78% line coverage**, and **73.06% method coverage**, surpassing the best 1,000-request baseline results of **34.90%**, **62.38%**, and **67.24%**, respectively. Moreover, LOGIAGENT remains competitive even against tools running for one hour (denoted with “-1h”), which typically send tens to hundreds of thousands of requests. Despite using significantly fewer requests,

Table 4: Operation Coverage by Different Tools

Services	LOGIAGENT	ARAT-RL		EvoMaster		Morest		RESTler	
	1000	1000	1h	1000	1h	1000	1h	1000	1h
Petstore	17	15	15	14	16	13	17	1	18
Bill-Service	17	3	7	2	12	4	14	0	1
Genome-Nexus	23	21	22	20	23	11	23	2	13
Features-Service	18	10	17	7	7	9	14	4	16
RESTCountries	22	6	14	17	22	8	18	2	22
News-Service	7	2	3	2	3	2	2	2	7
SCS	11	11	11	11	11	11	11	4	11
NCS	6	6	6	6	6	6	6	5	6
LanguageTool	2	1	2	1	1	1	1	2	2
Person-Controller	8	4	5	12	12	7	7	1	12
Project-Track	34	25	33	27	46	50	52	1	47
User-Management	14	8	17	12	12	13	16	1	13
Total: 231	179	112	152	131	171	135	181	25	168

**Figure 6: Average Code Coverage by Different Tools**

LOGIAGENT achieves comparable or better performance. For example, its branch coverage (39.98%) closely matches the highest one-hour result (40.83%), while its line (71.78%) and method (73.06%) coverage exceed most one-hour baselines, with only EvoMaster-1h achieving similar results (65.92% and 67.96%).

As noted in RQ1, LanguageTool is a particularly interesting case where LOGIAGENT achieves substantially higher code coverage than other baselines. This advantage is likely due to LLMs’ strong language modeling and processing capabilities, enabling LOGIAGENT to generate diverse linguistic inputs and effective oracles for testing LanguageTool’s behavior.

4.5 RQ4: Ablation Study

4.5.1 Method. The ablation study evaluates the impact of the Execution Memory component on LOGIAGENT’s code coverage. Specifically, we compare LOGIAGENT with two variants that omit key retrieval mechanisms: (i) *w/o params*: The API Request Executor does not retrieve relevant parameters from Execution Memory. (ii) *w/o refls*: The API Request Executor does not retrieve failure reflections from Execution Memory.

4.5.2 Results. Table 5 presents the ablation study results, showing that retrieval mechanisms in Execution Memory enhance code coverage by expanding the exploration space.

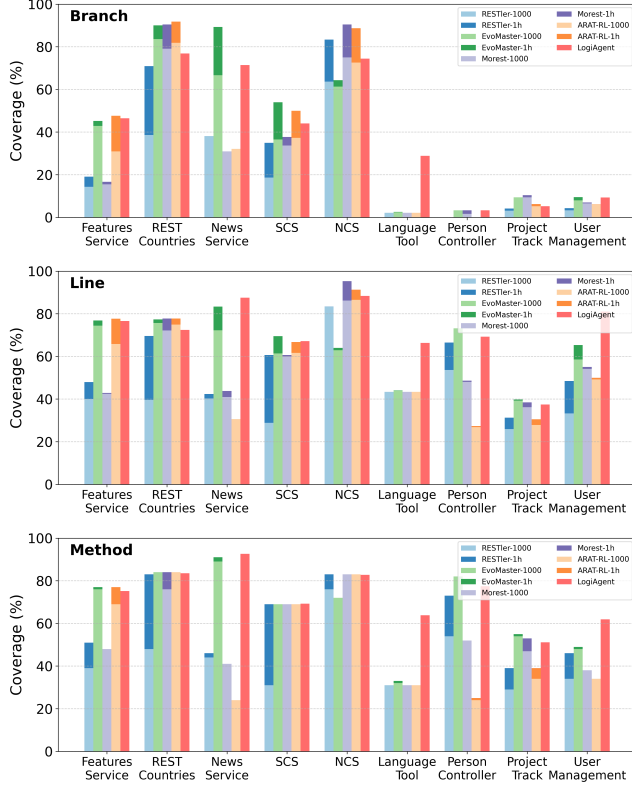


Figure 7: Code Coverage Comparison in Terms of Branch, Line, and Method

Table 5: Results of the Ablation Study.

Approaches	Branch	Line	Method
LOGIAGENT	39.96%	71.75%	73.04%
- w/o params	35.80% (-10.42%)	70.76% (-1.38%)	71.67% (-1.87%)
- w/o refs	37.72% (-5.60%)	71.03% (-1.00%)	71.84% (-1.64%)

5 DISCUSSION

5.1 Limitations

As an initial step towards logical testing for REST systems, LOGIAGENT shows promising potential, but its detection accuracy needs further improvement. False positives often result from LLM hallucinations when interpreting domain knowledge, understanding API specifications, and generating oracles. While LLMs can leverage general domain knowledge and common practices learned from public corpora, they still struggle with specific domains, especially those involving complex business logic. This is evident in the case of LanguageTool, where LOGIAGENT performs effectively despite the system’s 677K LOC and only 2 APIs, as language analysis and processing are core capabilities of LLMs. However, for other domain-specific systems (e.g., Bill-Service), LOGIAGENT exhibits higher false-positive rates. Future improvements could involve enhancing LOGIAGENT by constructing domain-specific knowledge bases, integrating not only API documentation but also the system’s code implementation (if available).

5.2 Threats to Validity

Internal validity is challenged by human annotation of the reported logical issues. To mitigate this, we involve two experts with over 5 years of development experience in REST systems. The experts collaboratively resolve any conflicts in their annotations to reach consensus. Another internal validity concern is the flaky nature of REST systems, influenced by network stability. To address this, we deploy most of the tested systems locally and include three online systems to ensure a diverse range of scenarios are covered.

External validity primarily concerns the generalizability of our findings to a broader range of REST systems. To address this, we use widely-tested real-world systems across various application domains and include online systems to ensure that the testing reflects practical, real-world scenarios.

6 RELATED WORK

In recent years, REST API has been widely used to build modern software systems. To guarantee the reliability of these systems, automated REST API testing has received extensive attention from researchers [16, 24, 41].

RESTler [3] and EvoMaster [9] constitute early contributions in automated REST API testing and have been used in some industrial systems. RESTler [3] performs stateful testing of REST APIs by analyzing the producer-consumer relationship between request types. EvoMaster [9] is a search-based system testing tool that uses evolutionary algorithms to generate test cases. It supports black-box testing and white-box testing of multiple Web APIs, such as REST, GraphQL, and RPC. Several subsequent studies focus on mining and using operation or parameter constraints in REST APIs to guide the testing process. Some studies use graphs to model the dependencies between operations to guide the testing process [27, 38]. RestTestGen [38] identifies data dependencies between operations to construct an operation dependency graph, and craft call sequences based on the graph. Morest [27] extends the operation dependency graph to a property graph and dynamically updates it. Besides, some studies use parameter constraints to guide parameter generation [20, 29, 40]. RESTest [29] uses IDL-Reasoner to extract inter-parameter constraints for each API and uses the constraint solver to automatically generate valid test cases. NLPTOREST [20] then uses natural language processing models to automatically identify inter-parameter constraints. Some researches have adopted reinforcement learning in automated REST API testing. ARAT-RL [21] uses Q-Learning to determine the priority of different operations and parameters during testing process. DeepRest [5] combines deep reinforcement learning and curiosity-driven learning to explore the optimal call sequence exploration strategy and parameter generation strategy.

Recently, LLMs are widely used in software engineering tasks [10, 18, 39]. Some researchers have explored adopting LLMs in REST API testing. RestGPT [22] enhances the REST API specifications by using LLMs to identify parameter constraints and generate parameter values. AutoRestTest [23] uses few-shot learning to prompt LLMs to generate valid parameter values. RESTSpecIT [6] leverages LLMs to analyze the API response to automatically infer the

API specification. These explorations focus on leveraging LLMs for API parameter value generation and API response comprehension but do not examine their potential to enhance the overall testing process, particularly logical testing.

For efficient detection of system issues, test oracle is an important part of automated REST API testing. Most of the existing work simply uses HTTP status codes as test oracle to validate the test result [3, 5, 9, 15, 21, 27, 29, 40], which limits their ability in issue detection. Dredd [8] validates test result based on the schema of payloads by using the Gavel library [13]. AGORA [2] uses Daikon to infer invariant properties from historical requests and responses, and uses them to validate the test result. However, all of the existing works can only be used for validation of the test results of single REST API. To the best of our knowledge, there is no existing work on testing and validation of logic issues in REST systems.

7 CONCLUSION

This paper introduces LOGIAGENT, a novel approach to logical testing for REST systems that overcomes the limitations of existing methods focused primarily on 500 server errors. By utilizing an LLM-driven multi-agent framework, LOGIAGENT validates API responses with logical oracles grounded in business logic. Experiments across 12 real-world systems demonstrate LOGIAGENT's superior performance in detecting both logical issues and **server crashes**, outperforming state-of-the-art tools. Our future work will aim to enhance LOGIAGENT's capabilities by incorporating domain-specific knowledge for tested REST systems.

REFERENCES

- [1] 2025. Replication Package for LogiAgent. <https://anonymous.4open.science/r/LogiAgent-5055/> Anonymous Replication repository for double-blind review.
- [2] Juan C. Alonso, Sergio Segura, and Antonio Ruiz-Cortés. 2023. AGORA: Automated Generation of Test Oracles for REST APIs. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 1018–1030. doi:10.1145/3597926.3598114
- [3] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. RESTler: stateful REST API fuzzing. In *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019*. IEEE / ACM, 748–758. doi:10.1109/ICSE.2019.00083
- [4] Hybrid Cloud. 2011. The nist definition of cloud computing. *National institute of science and technology, special publication* 800, 2011 (2011), 145.
- [5] Davide Corradini, Zeno Montolli, Michele Pasqua, and Mariano Ceccato. 2024. DeepREST: Automated Test Case Generation for REST APIs Exploiting Deep Reinforcement Learning. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*. ACM, 1383–1394. doi:10.1145/3691620.3695511
- [6] Alix Decrop, Gilles Perrouin, Mike Papadakis, Xavier Devroey, and Pierre-Yves Schobbens. 2024. You Can REST Now: Automated Specification Inference and Black-Box Testing of RESTful APIs with Large Language Models. *CoRR abs/2402.05102* (2024). doi:10.48550/ARXIV.2402.05102 arXiv:2402.05102
- [7] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. 2024. A Survey on In-context Learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 1107–1128. doi:10.18653/v1/2024.emnlp-main.64
- [8] Dredd. 2025. Dredd. Retrieved Mar 03, 2025 from <https://github.com/apiaryio/dredd>
- [9] EvoMaster. 2025. EvoMaster. Retrieved Mar 03, 2025 from <https://github.com/WebFuzzing/EvoMaster>
- [10] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. 31–53. doi:10.1109/ICSE-FoSE59343.2023.00008
- [11] R. Fielding, M. Nottingham, and J. Reschke. 2022. RFC 9110: HTTP Semantics.
- [12] Roy T Fielding and Richard N Taylor. 2002. Principled design of the modern web architecture. *ACM Transactions on Internet Technology (TOIT)* 2, 2 (2002), 115–150.
- [13] Gavel. 2025. Gavel. Retrieved Mar 03, 2025 from <https://github.com/apiaryio/gavel.js>
- [14] Genome Nexus. [n. d.]. Genome Nexus API. <https://www.genomenexus.org> Accessed: 2025-03-01.
- [15] Patrice Godefroid, Bo-Yuan Huang, and Marina Polishchuk. 2020. Intelligent REST API data fuzzing. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*. ACM, 725–736. doi:10.1145/3368089.3409719
- [16] Amid Golmohammadi, Man Zhang, and Andrea Arcuri. 2024. Testing RESTful APIs: A Survey. *ACM Trans. Softw. Eng. Methodol.* 33, 1 (2024), 27:1–27:41. doi:10.1145/3617175
- [17] Dominique Guinard, Vlad Trifa, and Erik Wilde. 2010. A resource oriented architecture for the web of things. In *2010 Internet of Things (IOT)*. IEEE, 1–8.
- [18] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (Dec. 2024), 79 pages. doi:10.1145/3695988
- [19] JaCoCo. 2025. JaCoCo Java Code Coverage Library. Retrieved Mar 5, 2025 from <https://www.eclemma.org/jacoco/>
- [20] Myeongsoo Kim, Davide Corradini, Saurabh Sinha, Alessandro Orso, Michele Pasqua, Rachel Tzoref-Brill, and Mariano Ceccato. 2023. Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*. ACM, 1232–1243. doi:10.1145/3597926.3598131
- [21] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2023. Adaptive REST API Testing with Reinforcement Learning. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 446–458. doi:10.1109/ASE56229.2023.00218
- [22] Myeongsoo Kim, Tyler Stennett, Dhruv Shah, Saurabh Sinha, and Alessandro Orso. 2024. Leveraging Large Language Models to Improve REST API Testing. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, NIER@ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 37–41. doi:10.1145/3639476.3639769
- [23] Myeongsoo Kim, Tyler Stennett, Saurabh Sinha, and Alessandro Orso. 2024. A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs. *CoRR abs/2411.07098* (2024). doi:10.48550/ARXIV.2411.07098 arXiv:2411.07098
- [24] Myeongsoo Kim, Qi Xin, Saurabh Sinha, and Alessandro Orso. 2022. Automated test generation for REST APIs: no time to rest yet. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual, South Korea) (ISSTA 2022)*. Association for Computing Machinery, New York, NY, USA, 289–301. doi:10.1145/3533767.3534401
- [25] Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. 2021. A Systematic Review of API Evolution Literature. *ACM Comput. Surv.* 54, 8, Article 171 (Oct. 2021), 36 pages. doi:10.1145/3470133
- [26] Li Li, Wu Chou, Wei Zhou, and Min Luo. 2016. Design Patterns and Extensibility of REST API for Networking Applications. *IEEE Transactions on Network and Service Management* 13, 1 (2016), 154–167. doi:10.1109/TNSM.2016.2516946
- [27] Yi Liu, Yuekang Li, Gelei Deng, Yang Liu, Ruiyuan Wan, Runchao Wu, Dandan Ji, Shiheng Xu, and Minli Bao. 2022. Morest: Model-based RESTful API Testing with Execution Feedback. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1406–1417. doi:10.1145/3510003.3510133
- [28] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2019. Test coverage criteria for RESTful web APIs. In *Proceedings of the 10th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation (Tallinn, Estonia) (A-TEST 2019)*. Association for Computing Machinery, New York, NY, USA, 15–21. doi:10.1145/3340433.3342822
- [29] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2020. RESTest: Black-Box Constraint-Based Testing of RESTful Web APIs. In *Service-Oriented Computing - 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14-17, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12571)*. Springer, 459–475. doi:10.1007/978-3-030-65310-1_33
- [30] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 11048–11064. doi:10.18653/v1/2022.emnlp-main.759
- [31] Andy Neumann, Nuno Laranjeiro, and Jorge Bernardino. 2021. An Analysis of Public REST Web Service APIs. *IEEE Transactions on Services Computing* 14, 4 (2021), 957–970. doi:10.1109/TSC.2018.2847344
- [32] Sam Newman. 2021. *Building microservices: designing fine-grained systems*. "O'Reilly Media, Inc."
- [33] OpenAI. [n. d.]. New embedding models with lower pricing. <https://openai.com/index/new-embedding-models-and-api-updates/>
- [34] Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.* 3 (2009), 333–389. <https://api.semanticscholar.org/CorpusID:207178704>
- [35] Swagger. [n. d.]. Swagger Petstore API. <https://petstore.swagger.io/v2> Accessed: 2025-03-01.
- [36] Chung-Hsuan Tsai, Shi-Chun Tsai, and Shih-Kun Huang. 2021. REST API Fuzzing by Coverage Level Guided Blackbox Testing. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. 291–300. doi:10.1109/QRS54544.2021.00040
- [37] UK Parliament. [n. d.]. Bills API. <https://bills-api.parliament.uk/api/v1> Accessed: 2025-03-01.
- [38] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. RESTTEST-GEN: Automated Black-Box Testing of RESTful APIs. In *13th IEEE International Conference on Software Testing, Validation and Verification, ICST 2020, Porto, Portugal, October 24-28, 2020*. IEEE, 142–152. doi:10.1109/ICST46399.2020.00024
- [39] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208
- [40] Huayao Wu, Lixin Xu, Xintao Niu, and Changhai Nie. 2022. Combinatorial Testing of RESTful APIs. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 426–437. doi:10.1145/3510003.3510151
- [41] Man Zhang and Andrea Arcuri. 2023. Open Problems in Fuzzing RESTful APIs: A Comparison of Tools. *ACM Trans. Softw. Eng. Methodol.* 32, 6, Article 144 (Sept. 2023), 45 pages. doi:10.1145/3597205
- [42] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Transactions on Software Engineering* 47, 2 (2021), 243–260. doi:10.1109/TSE.2018.2887384